

TILE GRID SYSTEM

OVERVIEW OF TILE GRID SYSTEM

The Tile Grid System is the central system that manages how tiles are placed on the grid. The main scripts behind the functionality of the Grid System are GridManager.cs and GridChunk.cs. Their functionality is pretty closely related. For information on Tile Placement, see “Tile_Placement_System.pdf”.

HOW THE GRID SYSTEM WORKS

Each of the ground area / grid chunk in the game gets its own instance of the GridChunk class. Each instance of the GridChunk class holds a 2D-array of GridCells, “GridCell[][] positionsOfCells”, which stores all the GridCells on the Grid Chunk. Each instance of the GridCell class holds a list of all the tiles on that grid cell (max one terrain tile and one object tile). The GridManager class, which is a singleton, keeps track of all of the GridChunks that are currently instantiated, using an array list called “gridChunks”. The GridManager and GridChunk classes each provide a public API that allows you to interact with the tiles on the GridChunks.

HOW TO USE THE API

In order to call a function of a given Grid Chunk, use a line like `GridManager.GM.gridChunks[chunk#].someFunction()`. Or, to access the active chunk, use `GridManager.GM.ActiveGridChunk.someFunction()`. As a shorthand, you can access most of the functions of the active grid chunk with simply `GridManager.GM.someFunction()`.

For functions that allow you to access specific tiles, you will sometimes have to provide the local integer grid coordinates of the tile. The cell at the center of each GridChunk starts at the coordinate `[0,0]`.

INTERNAL API

Public functions that can be called on a GridChunk mostly have to do with accessing or editing the tiles on that grid chunk. Below are a few of the most frequently used ones.

bool AtLeastOneTileIsOnChunk()

Returns true if any tiles are on the grid chunk, false if there are none.

Tile[] GetAllTilesOnGridChunk()

Returns an array of all the tiles on the grid chunk.

Tile[] GetAllTilesInRange(Vector2Int lower, Vector2Int upper)

Returns an array of all the tiles in the specified range. The two parameters specify the lower left bound and the upper right bound of the range using local grid coordinates.

void AddObject(GameObject objectToAdd)

Records that a tile has been added to the grid chunk. It finds the grid cell at the world coordinates of “objectToAdd” and adds the object to that grid cell.

void RemoveObject(GameObject objectToRemove)

Records that a tile has been removed from the grid chunk. It finds the grid cell at the world coordinates of “objectToRemove” and tells that grid cell to try to remove the object from it.

GameObject[] GetGameObjectsInGridCell(Vector2Int gridPosition)

Returns an array of the game objects of all of the tiles at the input grid position.

GameObject[] GetGameObjectsInGridCell(Vector3 worldPosition)

Returns an array of the game objects of all of the tiles at the input world position.

GameObject[] GetGameObjectsInGridCell(GameObject objectInCell)

Returns an array of the game objects of all of the tiles at the same location as the input object, “objectInCell”.

-Graydon Bruyn, May 2026